PLONK 코드베이스의 전체 구조 분석

1. 주요 모듈과 역할

PLONK 코드베이스는 다음과 같은 주요 모듈로 구성되어 있습니다:

1.1 최상위 모듈

- **src/lib.rs**: 라이브러리의 진입점으로, plonk-core와 plonk-hashing 모듈을 재내보내기(re-export)합니다.
- **plonk-core**: PLONK 증명 시스템의 핵심 구현을 포함합니다.
- **plonk-hashing**: 다양한 해싱 알고리즘(Poseidon 등)을 구현합니다.
- **plonk-book**: PLONK 알고리즘과 구현에 대한 상세한 설명을 제공하는 문서입니다.

1.2 plonk-core 모듈

plonk-core 모듈은 PLONK 증명 시스템의 핵심 구현을 포함하며, 다음과 같은 하위 모듈로 구성되어 있습니다:

- **circuit.rs**: 회로 정의와 관련된 트레이트와 함수를 제공합니다.
- **commitment.rs**: 다항식 약정 체계와 관련된 기능을 제공합니다.
- **constraint_system**: 회로의 제약 조건을 정의하고 관리하는 기능을 제공합니다.
- **error.rs**: 오류 처리와 관련된 기능을 제공합니다.
- **lookup.rs**: 룩업 테이블과 관련된 기능을 제공합니다.
- **permutation**: 순열 인수(permutation argument)와 관련된 기능을 제공합니다.
- **prelude.rs**: 자주 사용되는 타입과 함수를 재내보내기합니다.

- **proof_system**: 증명 생성 및 검증과 관련된 기능을 제공합니다.
- **transcript.rs**: Fiat-Shamir 변환을 위한 트랜스크립트 관리 기능을 제공합니다.
- **util.rs**: 유틸리티 함수를 제공합니다.

1.3 constraint_system 모듈

constraint_system 모듈은 회로의 제약 조건을 정의하고 관리하는 기능을 제공하며, 다음과 같은 하위 모듈로 구성되어 있습니다:

- **arithmetic.rs**: 산술 게이트와 관련된 기능을 제공합니다.
- **boolean.rs**: 불리언 게이트와 관련된 기능을 제공합니다.
- **composer.rs**: StandardComposer 구현을 포함합니다.
- **ecc**: 타원 곡선 연산과 관련된 기능을 제공합니다.
- **hash.rs**: 해시 함수와 관련된 기능을 제공합니다.
- **helper.rs**: 도우미 함수를 제공합니다.
- **logic.rs**: 논리 게이트와 관련된 기능을 제공합니다.
- **lookup.rs**: 룩업 게이트와 관련된 기능을 제공합니다.
- **range.rs**: 범위 게이트와 관련된 기능을 제공합니다.
- **variable.rs**: 변수 관리와 관련된 기능을 제공합니다.

1.4 proof_system 모듈

proof_system 모듈은 증명 생성 및 검증과 관련된 기능을 제공하며, 다음과 같은 하위 모듈로 구성되어 있습니다:

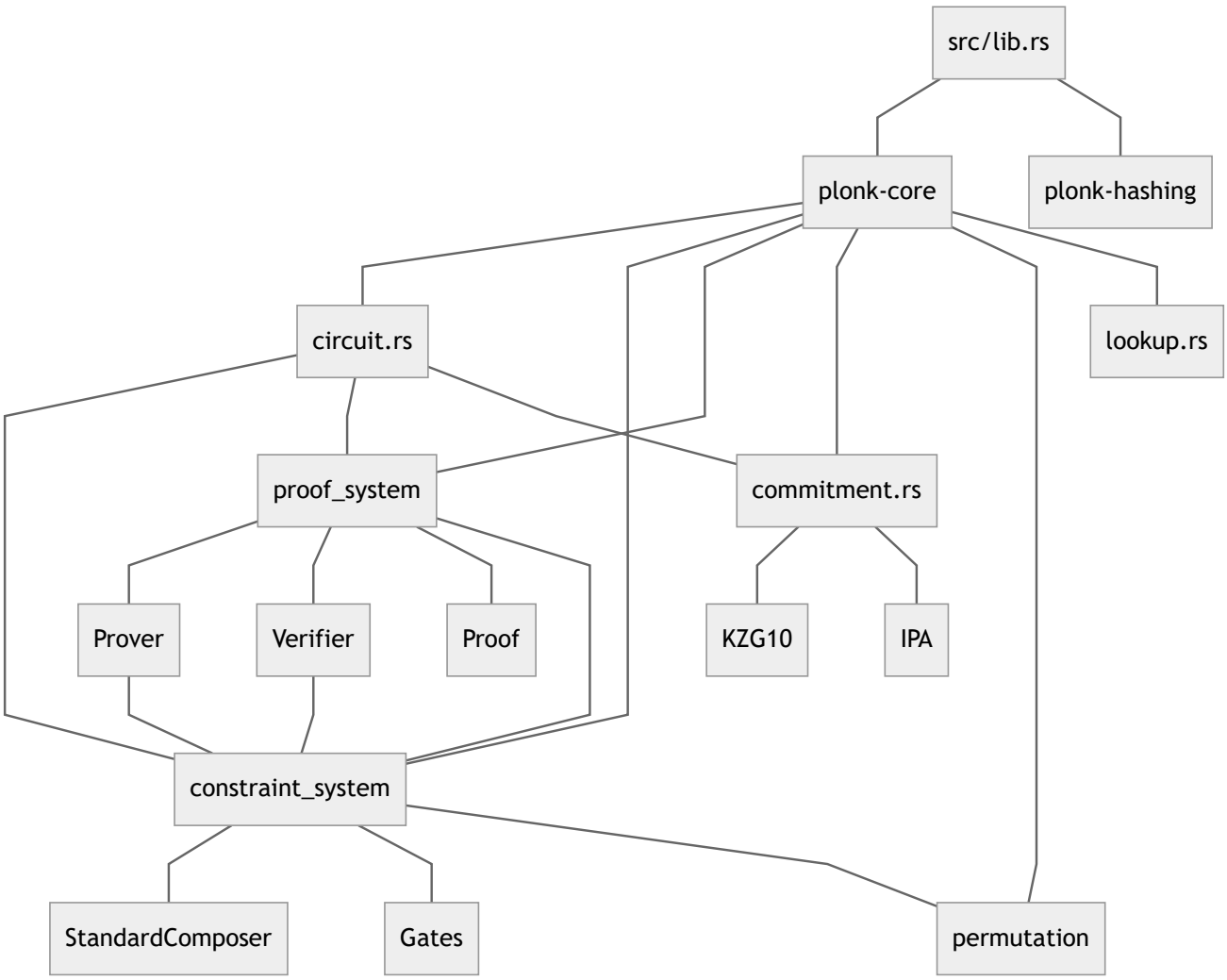
- **linearisation_poly.rs**: 선형화 다항식과 관련된 기능을 제공합니다.
- **permutation.rs**: 순열 인수와 관련된 기능을 제공합니다.
- **pi**: 공개 입력(public input)과 관련된 기능을 제공합니다.

- **preprocess.rs**: 전처리 단계와 관련된 기능을 제공합니다.
- **proof.rs**: 증명 구조체와 관련된 기능을 제공합니다.
- **prover.rs**: 증명자와 관련된 기능을 제공합니다.
- **quotient_poly.rs**: 몫 다항식과 관련된 기능을 제공합니다.
- **verifier.rs**: 검증자와 관련된 기능을 제공합니다.
- **widget.rs**: 위젯과 관련된 기능을 제공합니다.

2. 모듈 간의 의존성과 상호작용

PLONK 코드베이스의 모듈 간 의존성과 상호작용은 다음과 같습니다:

2.1 주요 의존성 흐름



2.2 주요 상호작용

1. **Circuit과 StandardComposer**: Circuit 트레이트는 StandardComposer를 사용하여 회로를 정의합니다. Circuit의 gadget 메서드는 StandardComposer를 매개변수로 받아 회로의 제약 조건을 추가합니다.

2. **StandardComposer와 Gates**: StandardComposer는 다양한 게이트 타입(산술, 범위, 논리 등)을 사용하여 회로의 제약 조건을 정의합니다.

3. **Prover와 StandardComposer**: Prover는 StandardComposer를 사용하여 회로의 제약 조건을 만족하는 증명을 생성합니다.

4. **Verifier와 StandardComposer**: Verifier는 StandardComposer를 사용하여 증명이 회로의 제약 조건을 만족하는지 검증합니다.

5. **Prover/Verifier와 Commitment**: Prover와 Verifier는 다항식 약정 체계(KZG10, IPA 등)를 사용하여 다항식에 대한 약정을 생성하고 검증합니다.

3. 핵심 데이터 구조와 목적

PLONK 코드베이스의 핵심 데이터 구조와 그 목적은 다음과 같습니다:

3.1 Variable

```
pub struct Variable(pub(crate) usize);
```

Variable은 회로 내의 변수를 나타내는 구조체입니다. 내부적으로는 단순한 정수 식별자를 래핑하고 있습니다. 이 식별자는 변수의 고유 ID로 사용됩니다.

3.2 StandardComposer

```
pub struct StandardComposer<F, P>
where
F: PrimeField,
P: ModelParameters<BaseField = F>,
{
        pub(crate) n: usize,
        pub(crate) q_m: Vec<F>,
        pub(crate) q_l: Vec<F>,
        pub(crate) q_r: Vec<F>,
        pub(crate) q_o: Vec<F>,
        pub(crate) q_4: Vec<F>,
```

```
        pub(crate) q_c: Vec<F>,
        pub(crate) q_hl: Vec<F>,
        pub(crate) q_hr: Vec<F>,
        pub(crate) q_h4: Vec<F>,
        pub(crate) q_arith: Vec<F>,
        pub(crate) q_range: Vec<F>,
        pub(crate) q_logic: Vec<F>,
        pub(crate) q_fixed_group_add: Vec<F>,
        pub(crate) q_variable_group_add: Vec<F>,
        pub(crate) q_lookup: Vec<F>,
        pub(crate) public_inputs: PublicInputs<F>,
        pub(crate) intended_pi_pos: Vec<usize>,
        pub(crate) w_l: Vec<Variable>,
        pub(crate) w_r: Vec<Variable>,
        pub(crate) w_o: Vec<Variable>,
        pub(crate) w_4: Vec<Variable>,
        pub(crate) lookup_table: LookupTable<F>,
        pub(crate) zero_var: Variable,
        pub(crate) variables: HashMap<Variable, F>,
        pub(crate) perm: Permutation,
        __: PhantomData<P>,
    }
```

StandardComposer는 회로 구성을 위한 주요 클래스입니다. 다음과 같은 주요 구성 요소를 포함합니다:

- **n**: 회로의 게이트 수
- **q_m, q_l, q_r, q_o, q_4, q_c**: 게이트 선택자 벡터
- **q_hl, q_hr, q_h4**: 고차 선택자 벡터(Poseidon 해시 등에 사용)
- **q_arith, q_range, q_logic, q_fixed_group_add, q_variable_group_add, q_lookup**: 게이트 타입 선택자 벡터
- **public_inputs**: 공개 입력
- **w_l, w_r, w_o, w_4**: 와이어 벡터
- **lookup_table**: 룩업 테이블
- **zero_var**: 0 값을 가진 변수
- **variables**: 변수 값 맵
- **perm**: 순열 인수

3.3 Proof

```
pub  struct Proof<F, PC>
where
F: PrimeField,
PC: HomomorphicCommitment<F>,
{
pub(crate) a_comm: PC::Commitment,
pub(crate) b_comm: PC::Commitment,
pub(crate) c_comm: PC::Commitment,
pub(crate) d_comm: PC::Commitment,
```

```
    pub(crate) z_comm: PC::Commitment,

    pub(crate) t_1_comm: PC::Commitment,

    pub(crate) t_2_comm: PC::Commitment,

    pub(crate) t_3_comm: PC::Commitment,

    pub(crate) t_4_comm: PC::Commitment,

    pub(crate) a_eval: F,

    pub(crate) b_eval: F,

    pub(crate) c_eval: F,

    pub(crate) d_eval: F,

    pub(crate) a_next_eval: F,

    pub(crate) b_next_eval: F,

    pub(crate) d_next_eval: F,

    pub(crate) s_sig1_eval: F,

    pub(crate) s_sig2_eval: F,

    pub(crate) z_eval: F,

    pub(crate) z_next_eval: F,

    pub(crate) t_eval: F,

    pub(crate) r_eval: F,

    pub(crate) openings: PC::Proof,

    pub(crate) batch_proof: Option<PC::BatchProof>,

    }
```

Proof는 생성된 증명을 나타내는 구조체입니다. 다음과 같은 주요 구성 요소를 포함합니다:

- **a_comm, b_comm, c_comm, d_comm**: 와이어 다항식에 대한 약정
- **z_comm**: 순열 다항식에 대한 약정
- **t_1_comm, t_2_comm, t_3_comm, t_4_comm**: 몫 다항식에 대한 약정
- **a_eval, b_eval, c_eval, d_eval**: 와이어 다항식의 평가값
- **a_next_eval, b_next_eval, d_next_eval**: 다음 와이어 다항식의 평가값
- **s_sig1_eval, s_sig2_eval**: 순열 인수의 평가값
- **z_eval, z_next_eval**: 순열 다항식의 평가값
- **t_eval**: 몫 다항식의 평가값
- **r_eval**: 선형화 다항식의 평가값
- **openings**: 다항식 개방(opening) 증명
- **batch_proof**: 배치 증명

3.4 ProverKey

```
pub  struct ProverKey<F: PrimeField> {
pub(crate) q_m: DensePolynomial<F>,
pub(crate) q_l: DensePolynomial<F>,
pub(crate) q_r: DensePolynomial<F>,
pub(crate) q_o: DensePolynomial<F>,
pub(crate) q_c: DensePolynomial<F>,
pub(crate) q_4: DensePolynomial<F>,
pub(crate) q_hl: DensePolynomial<F>,
pub(crate) q_hr: DensePolynomial<F>,
pub(crate) q_h4: DensePolynomial<F>,
```

```
    pub(crate) q_arith: DensePolynomial<F>,
    pub(crate) q_range: DensePolynomial<F>,
    pub(crate) q_logic: DensePolynomial<F>,
    pub(crate) q_lookup: DensePolynomial<F>,
    pub(crate) q_fixed_group_add: DensePolynomial<F>,
    pub(crate) q_variable_group_add: DensePolynomial<F>,
    pub(crate) sigma_1: DensePolynomial<F>,
    pub(crate) sigma_2: DensePolynomial<F>,
    pub(crate) sigma_3: DensePolynomial<F>,
    pub(crate) sigma_4: DensePolynomial<F>,
    pub(crate) linear_eval: F,
    pub(crate) permutation_eval: F,
    pub(crate) lookup_eval: F,
    pub(crate) table_eval: F,
    pub(crate) wire_eval: F,
    pub(crate) range_eval: F,
    pub(crate) logic_eval: F,
    pub(crate) fixed_group_add_eval: F,
    pub(crate) variable_group_add_eval: F,
    pub(crate) public_eval: F,
    pub(crate) lagrange_1_eval: F,
    pub(crate) lagrange_n_eval: F,
    pub(crate) domain: EvaluationDomain<F>,
```

```
pub(crate) lookup_table: LookupTable<F>,

pub(crate) n: usize,

}
```

ProverKey는 증명자가 증명을 생성하는 데 필요한 키입니다. 다음과 같은 주요 구성 요소를 포함합니다:

- **q_m, q_l, q_r, q_o, q_c, q_4**: 게이트 선택자 다항식
- **q_hl, q_hr, q_h4**: 고차 선택자 다항식
- **q_arith, q_range, q_logic, q_lookup, q_fixed_group_add, q_variable_group_add**: 게이트 타입 선택자 다항식
- **sigma_1, sigma_2, sigma_3, sigma_4**: 순열 다항식
- **linear_eval, permutation_eval, lookup_eval, ...**: 다양한 평가값
- **domain**: 평가 도메인
- **lookup_table**: 룩업 테이블
- **n**: 회로의 게이트 수

3.5 VerifierKey

```
pub  struct VerifierKey<F, PC>

where

F: PrimeField,

PC: HomomorphicCommitment<F>,

{

pub(crate) q_m: PC::Commitment,
```

```
    pub(crate) q_l: PC::Commitment,

    pub(crate) q_r: PC::Commitment,

    pub(crate) q_o: PC::Commitment,

    pub(crate) q_c: PC::Commitment,

    pub(crate) q_4: PC::Commitment,

    pub(crate) q_hl: PC::Commitment,

    pub(crate) q_hr: PC::Commitment,

    pub(crate) q_h4: PC::Commitment,

    pub(crate) q_arith: PC::Commitment,

    pub(crate) q_range: PC::Commitment,

    pub(crate) q_logic: PC::Commitment,

    pub(crate) q_lookup: PC::Commitment,

    pub(crate) q_fixed_group_add: PC::Commitment,

    pub(crate) q_variable_group_add: PC::Commitment,

    pub(crate) sigma_1: PC::Commitment,

    pub(crate) sigma_2: PC::Commitment,

    pub(crate) sigma_3: PC::Commitment,

    pub(crate) sigma_4: PC::Commitment,

    pub(crate) linear_eval: F,

    pub(crate) permutation_eval: F,

    pub(crate) lookup_eval: F,

    pub(crate) table_eval: F,

    pub(crate) wire_eval: F,
```

```
pub(crate) range_eval: F,

pub(crate) logic_eval: F,

pub(crate) fixed_group_add_eval: F,

pub(crate) variable_group_add_eval: F,

pub(crate) public_eval: F,

pub(crate) lagrange_1_eval: F,

pub(crate) lagrange_n_eval: F,

pub(crate) n: usize,

}
```

VerifierKey는 검증자가 증명을 검증하는 데 필요한 키입니다. ProverKey와 유사하지만, 다항식 대신 약정을 포함합니다.

3.6 PublicInputs

```
pub  struct PublicInputs<F: PrimeField> {

pub(crate) pi_pos: HashMap<usize, F>,

}
```

PublicInputs는 공개 입력을 나타내는 구조체입니다. 위치와 값의 맵으로 구성됩니다.